ack3

SECURITY PARTNER

CoW

Composable Poller

ENGAGEMENT	SECURITY REVIEW
REVISION	2.1
DELIVERED	2026-09-29
LEAD AUDITOR	Michal Převrátil

ACK3 LTD / 2026

Contents

1. Document Revisions 4
2. Overview 5
2.1. ack3 5
2.2. Review Methodology 6
2.3. Finding Classification 7
2.4. Review Team 9
2.5. Disclaimer 9
3. Executive Summary 10
Revision 1.0 10
Revision 1.1 11
Revision 2.0 11
Revision 2.1 12
4. Findings Summary 13
Report Revision 1.0 14
Revision Team 14
System Overview 14
Trust Model 15
Findings 16
Report Revision 1.1 21
Revision Team 21
Report Revision 2.0 22
Revision Team 22
System Overview 22
Trust Model 22
Report Revision 2.1 23
Revision Team 23

Appendix A: How to cite 24

1. Document Revisions

1.0	Final Report	September 4, 2026
1.1	Fix Review	September 9, 2026
2.0	Final Report	September 24, 2026
2.1	Commit Verification	September 29, 2026

2. Overview

This document presents the findings of our review.

2.1. ack3

ack3 is an in-house team of security researchers performing audits focused on manual code reviews with extensive fuzz testing and AI-assisted workflows for Ethereum and Solana. The ack3 team is trusted by top-tier organizations in web3, including Lido, Safe, and Axelar.

We develop the open-source security and developer tooling Wake, which powers the fuzzing, static analysis, and graph-based review on our audits and helps auditors discover hardly recognizable edge-case vulnerabilities.

Our team teaches blockchain security at the Czech Technical University in Prague, led by founder Josef Gattermayer, Ph.D.

For ack3, security is a long-term partnership: we work alongside our clients from design through post-deployment.

2.2. Review Methodology

We begin each review by mapping the codebase and the interactions between contracts and external components using Wake's graph tooling. We work with the provided documentation and materials to discover discrepancies between the expected behavior and the actual implementation. Where applicable, we build a mathematical model of the protocol to cross-compare with the code and identify subtle bugs and inconsistencies.

At the start of the review, we launch an automated ack3 AI scan pinned to the scoped commits, running in parallel with the manual review. The scan provides additional coverage and new leads for the team to analyze; each scan finding is manually reviewed by the team.

During the review, we apply AI-assisted workflows and skills that support codebase onboarding and navigation, combined with Wake's static analyzer and language server for context-aware code analysis.

We launch AI agents to build on our vulnerability hypotheses and expand the attack surface with new escalation paths. All findings are manually reviewed and validated by the team. Where applicable, we confirm findings and verify the exact behavior of the code with unit tests and fuzzing. Where the engagement includes a fuzzing campaign, it is manually guided by an auditor; the fuzz tests themselves are AI-generated.

Once sufficient context is gathered, we search for deeper, chained issues based on the attack vectors discovered during review. This includes combining smaller weaknesses, testing trust-boundary assumptions, and evaluating whether unexpected behavior can lead to practical impact.

The validated results are documented in the report: findings with severity, affected components, impact, and recommended remediation.

2.3. Finding Classification

The Severity of each finding combines two sub-ratings, Impact and Likelihood, and ranges from Info to Critical.

If we find a scenario in which an issue is exploitable, it is assigned an impact rating of High, Medium, or Low, based on its consequences for the system. If we find no way to exploit it, or the issue is only exploitable given a change in configuration (system settings or parameters, such as deployment scripts, compiler configurations, or multi-signature wallets for owners) or in the codebase, it is assigned Warning or Info.

Low to High impact issues also have a Likelihood, which measures the probability of exploitation at runtime.

Severity

		LIKELIHOOD			
		HIGH	**MEDIUM**	**LOW**	**N/A**
IMPACT	**HIGH**	Critical	High	Medium	-
	MEDIUM	High	Medium	Low	-
	LOW	Medium	Low	Low	-
	WARNING	-	-	-	Warning
	INFO	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue leads to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue results in serious consequences.
- **Low** - Code that activates the issue has consequences that are recoverable or do not jeopardize the system's normal operation.
- **Warning** - The issue cannot be exploited given the current code or `configuration`, but could become a security vulnerability if either changed. If we have not found a way to exploit the issue given the time constraints, it may be marked as `Warning` or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations, or issues that would become security-related if code or `configuration` changed.

Likelihood

- **High** - The issue is exploitable by virtually anyone under any circumstance.
- **Medium** - Exploiting the issue requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

For a revision's authors, see the "Revision team" section in that revision's chapter.

NAME	POSITION
Michal Převrátil	Lead Auditor
Naoki Yoshida	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We have made our best effort to find all vulnerabilities in the system; however, our findings should not be considered a complete list of issues. Statements in this document should not be interpreted as investment or legal advice, nor should the authors be held accountable for decisions based on them.

3. Executive Summary

CoW is a DEX aggregation protocol that settles user orders in batch auctions executed by competing solvers, with the ComposableCoW framework extending it to conditional orders whose parts become tradeable over time. Composable Poller funds these conditional orders just in time, transferring the sell amount from the funder to the order owner only once a part is ready to trade.

Revision 1.0

CoW engaged ack3 to perform a security review of CoW Composable Poller with a total time donation of 2 engineering days over a period between August 31 and September 4, 2026, with Michal Převrátil as the lead auditor.

The review was performed on the commit `3ff3cf5`[1] in the composable-cow repository, and the scope was the `src/types/ComposableCowPoller.sol` file.

We began our review by preparing a differential, manually guided fuzz test in the Wake framework, modeling the expected behavior of the contract and comparing it to the actual behavior. We then proceeded with a deep manual review of the code. At the end of the review, we launched an automated AI scan and reviewed all generated vulnerability hypotheses.

During the review, we focused in particular on the following aspects:

- the signature verification captures all required fields;
- the register and revoke actions are properly validated for authorization;
- the contract is protected against replay attacks;
- the contract correctly integrates with the other CoW contracts;
- EIP-712 is implemented consistently with the specification; and
- the documentation is accurate and users are informed about the trust

boundaries.

Our review resulted in a single informational finding, I1. The finding describes inaccurate documentation of the trust boundaries between the `ComposableCowPoller` contract in scope and the rest of the CoW system. The code quality exceptionally good.

ack3 recommends CoW to:

- clearly document which CoW handlers are supported by the poller;
- clearly document the trust assumptions for the handler; and
- consider whitelisting only the first-party supported handlers.

See Report Revision 1.0 for the system overview and trust model.

Revision 1.1

CoW engaged ack3 to perform a fix review of the finding identified in the previous revision.

The review was performed on September 9, 2026, on commit 665ee6d[2].

The previously reported finding I1 was confirmed to be fixed. No new findings were identified during the review.

Revision 2.0

CoW engaged ack3 to perform a security review of the pull request PR#84 in the cowdao-grants/cow-shed repository with a total time donation of 1 engineering day over a period between September 22 and September 24, 2026, with Michal Převrátil as the lead auditor.

The scope covered changes to the `src/COWShedFactory.sol` file from commit 5eed9fa[3] through commit 24cad12[4], inclusive.

The manual review was accompanied by an automated AI scan, which also covered interactions with perimeter contracts.

No findings were reported, but the review led to an update to the trust model. See Report Revision 2.0 for details.

Revision 2.1

CoW engaged ack3 to compare the commits reviewed in previous revisions with the corresponding commits on the main branches after the pull requests were squashed and merged.

The commit 665ee6d[5] reviewed in revision 1.1 corresponded to the squashed commit bc4d5f6[6] on the main branch of the cowprotocol/composable-cow repository. The two commits differed in `README.md` and the NatSpec comment in `src/types/ComposableCowPoller.sol`.

The commit 24cad12[7] reviewed in revision 2.0 and the squashed commit d28a3f3[8] on the main branch of the cowdao-grants/cow-shed repository had identical repository trees.

No new code was reviewed.

[1] full commit hash: `3ff3cf519f2b166713b8c5ff309bde14d6f0dff4`, link to commit

[2] full commit hash: `665ee6d38176c509da5356c3d74285b3f8f2b99d`, link to commit

[3] full commit hash: `5eed9faf3e57687ad087ed543602e3d85a3bf23a`, link to commit

[4] full commit hash: `24cad126b18e4c6bebc3b8f74758c6671339fc08`, link to commit

[5] full commit hash: `665ee6d38176c509da5356c3d74285b3f8f2b99d`, link to commit

[6] full commit hash: `bc4d5f66fe044fdd8fbc78c9cf6ae99158f06a4c`, link to commit

[7] full commit hash: `24cad126b18e4c6bebc3b8f74758c6671339fc08`, link to commit

[8] full commit hash: `d28a3f3541473fd75bc338cf9d28fca49a90321f`, link to commit

4. Findings Summary



Figure 1. Findings by Severity

FINDING TITLE	SEVERITY	REVISION	STATUS
I1: Inaccurate poller documentation	INFO	1.0	Fixed

Table 2. Table of Findings

Report Revision 1.0

Revision Team

NAME	POSITION
Michal Převrátil	Lead Auditor
Naoki Yoshida	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

`ComposableCowPoller` provides just-in-time funding for ComposableCoW conditional orders, keeping the sell amount with the funder until each part is ready to trade. A funder registers a schedule that pins the conditional-order handler, the order owner, a salt, and the order's `staticInput`, and whose key is derived from the funder, handler, owner, and salt. Schedules are registered and revoked by the funder directly, by an EIP-712 signature that any relayer may submit before its deadline, or by the funder's own CowShed proxy resolved through the factory pinned at deployment. Registrations and delegated revocations carry an authorization epoch that must match storage; revocation clears the schedule and increments the epoch, invalidating earlier authorizations while leaving the key reusable. The permissionless `pollFunds`, typically called from a CoW Protocol pre-hook, re-derives the order's params hash, requires it to be authorized in `ComposableCoW.singleOrders`, and reads the current order from the handler. It then transfers the order's `sellAmount` from the funder to the owner through the funder's allowance and records the order digest, so each digest is funded only once.

Trust Model

The poller distinguishes two roles: the funder, which supplies the sell tokens, and the owner, which holds the conditional order in `ComposableCoW` and receives the pulled funds. They may be the same address. The contract holds no funds and grants itself no privileges; every transfer draws on the token allowance the funder grants it.

The funder fully trusts the owner. `pollFunds` moves the sell amount to the owner, and the poller places no constraint on those funds afterwards. The owner also authorizes and removes its own conditional orders in `ComposableCoW`, so removing an order pauses funding only while the same `paramsHash` stays unauthorized: reauthorizing it resumes funding without new funder consent, as long as the schedule and the allowance remain. The funder's exposure is therefore bounded by the granted allowance rather than by the state of the conditional order. Revoking the schedule stops further funding under it, but leaves the allowance in place for any schedule registered later.

The funder is responsible for selecting a compatible handler and configuring it in a compatible way. `pollFunds` takes the sell token, the sell amount, and the time validity from `getTradeableOrder`; it does not pin the token at registration, cap the amount, or check `validTo` after the handler returns. Replay protection is keyed to the exact order digest, so funding a part once holds only for handlers whose output is deterministic per part. A handler whose output depends on mutable state, such as an oracle price or a balance, yields a new digest whenever that state changes, and each digest is funded once. The `staticInput` stored in the schedule is neither part of the schedule key nor validated at registration; a value that does not match the authorized order makes `pollFunds` revert `OrderNotLive`. These boundaries are documented inaccurately, which is the subject of I1.

`COMPOSABLE_COW` and `COW_SHED_FACTORY` are pinned at deployment and trusted

thereafter. The factory must derive proxies from the owner alone, because a call from `proxyOf(funder)` is what proves the funder authorized a `FromShed` action, and it must deploy sheds that forward ERC-1271 signatures, because a shed without forwarding can be funded but cannot settle. `pollFunds` is permissionless: any caller can trigger the transfer while the handler's window is open, so the funder does not control its timing.

Findings

This section lists the findings from this revision; for all findings, see the Findings Summary.

I1: Inaccurate poller documentation

Impact:	Info	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	Human
Target:	README.md, ComposableCowPoller.sol		

Description

The README and NatSpec document stronger or stale guarantees than `ComposableCowPoller` enforces. The implementation is internally consistent, but the current wording can mislead funders and integrators about the boundary provided by a registered schedule and a standing token allowance.

The README states that `pollFunds` verifies that the order is active, transfers only the current part, and funds each part once.

Listing 1. Excerpt from README

```
121 `ComposableCowPoller` enables an EOA to create a ComposableCoW conditional
    order, such as a TWAP, without moving the full sell amount into the
    order's smart account up front. The funds remain in the EOA until each
    part is ready to trade. A pre-hook then calls `pollFunds`, which verifies
    that the order is still active and transfers only that part's sell amount
    to the smart account. Each part can be funded only once.
```

`pollFunds` checks only that the registered conditional order hash is currently authorized in `ComposableCoW.singleOrders`, then asks the handler for the order. Replay protection is keyed to exact `orderDigest` reuse, and the transferred token and amount are taken from the returned order. For handlers whose output can change with mutable state, the accurate guarantee is per digest, not per conceptual part.

Listing 2. Excerpt from ComposableCowPoller.pollFunds

```
440 // The order must still be authorised; `remove` disables the poller.
```

```
441 if (!COMPOSABLE_COW.singleOrders(schedule.owner, paramsHash))
442     revert OrderNotLive();
```

Listing 3. Excerpt from ComposableCowPoller.pollFunds

```
445 GPv2Order.Data memory order = schedule.handler.getTradeableOrder(
446     schedule.owner,
447     address(this),
448     paramsHash,
449     schedule.staticInput,
450     bytes("")
```

Listing 4. Excerpt from ComposableCowPoller.pollFunds

```
454 bytes32 orderDigest = GPv2Order.hash(
455     order,
456     COMPOSABLE_COW.domainSeparator()
457 );
458 if (funded[id][orderDigest]) return false;
459 funded[id][orderDigest] = true;
```

Listing 5. Excerpt from ComposableCowPoller.pollFunds

```
461 order.sellToken.safeTransferFrom(
462     schedule.funder,
463     schedule.owner,
464     order.sellAmount
```

The README also says removing the conditional order stops further funding. That is true only while the same `paramsHash` remains unauthorized; if the owner reauthorizes it while the poller schedule and token allowance remain, `pollFunds` can fund again without new funder consent.

Listing 6. Excerpt from README

```
127 Removing the conditional order from ComposableCoW stops further funding.
    Revoking its funding schedule does too, but does not remove the funder's
    token allowance; revoke that allowance separately when it is no longer
    needed.
```

`ScheduleRegistered` describes an update or replacement path, but `_register` rejects an occupied schedule key. Replacement requires revocation and a new registration.

Listing 7. Excerpt from ComposableCowPoller

```
113 /// @notice Emitted when a schedule is registered or updated.
114 /// @param id The deterministic key of the schedule.
115 /// @param owner The conditional-order owner and pull destination.
116 /// @param funder The token source that registered the schedule.
117 /// @param authEpoch The schedule's authorization epoch.
118 /// @param paramsHash The ComposableCoW order key this schedule funds.
    `id` deliberately excludes
119 ///        `staticInput`, so re-registering the same funder, handler,
    owner, and salt replaces
120 ///        the stored schedule. Logging the hash names the order each
    registration points at,
```

Listing 8. Excerpt from ComposableCowPoller._register

```
263 if (schedules[id].funder != address(0)) revert AlreadyRegistered();
```

`funded` says replay history survives schedule updates, but active schedules are not updated. The history instead survives revocation and re-registration.

Listing 9. Excerpt from ComposableCowPoller

```
79 /// @dev `id => orderDigest => funded`. History survives schedule updates
   so an old order cannot be replayed.
```

`OnlyFunder` says it applies to registration, updates, and revocation, but the error is used only by direct registration.

Listing 10. Excerpt from ComposableCowPoller

```
82 /// @notice Thrown when someone other than the schedule funder registers,
   updates, or revokes a schedule.
83 error OnlyFunder();
```

Listing 11. Excerpt from ComposableCowPoller.register

```
195 if (msg.sender != schedule.funder) revert OnlyFunder();
```

The handler-window comment states a handler behavior as a poller invariant. `pollFunds` does not read `validTo` after `getTradeableOrder` returns, so the window holds only for handlers that revert outside it. The comment should present the window as a trust assumption on the handler to which `pollFunds` delegates time validity, not as a check the poller performs.

Listing 12. Excerpt from ComposableCowPoller.pollFunds

```
444 // The handler yields the current order and reverts outside its window.
445 GPv2Order.Data memory order = schedule.handler.getTradeableOrder(
```

Recommendation

Update the README and NatSpec to match the implemented invariants. State that `pollFunds` is permissionless, checks the registered schedule and `ComposableCoW.singleOrders`, delegates token, amount, and time validity to the handler, records funding per `orderDigest`, and only pauses after conditional-order removal until the order hash is reauthorized unless the schedule is also revoked. Also update the stale comments for schedule replacement, replay history, `OnlyFunder`, and handler window checks.

Fix 1.1

The README and NatSpec comments were updated to better reflect the trust model and current implementation.

Go back to Findings Summary

Report Revision 1.1

Revision Team

Revision team is the same as in Report Revision 1.0.

Report Revision 2.0

Revision Team

NAME	POSITION
Michal Převrátil	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The update added the `COWShedFactory.executeOwnHooks` function, which lets callers execute hooks without a signature. The function uses `msg.sender` for authorization, deriving the caller's shed address from it so hooks execute only on that caller's shed. It deploys the shed if needed, forwards any native tokens sent with the call, and executes the hooks in the same transaction. The `initializeProxy` function was also changed to return the shed address.

Trust Model

The `COWShedFactory.executeOwnHooks` function derives the caller's shed from `msg.sender` and executes the supplied `calls` without verifying a signature over them. If the shed owner is a contract, it must prevent untrusted parties from causing it to call the factory with arbitrary `calls`; otherwise, those parties can execute hooks on the owner's shed.

Report Revision 2.1

Revision Team

Revision team is the same as in Report Revision 2.0.

Appendix A: How to cite

Cite this document as:

> ack3, Security Review | CoW: Composable Poller, September 29, 2026.



Thank you.

COMPANY	ACK3 LTD
WEB	ack3.ai
X	@ack3ai
MAIL	hello@ack3.ai